



SOFT-11-C1 Proyecto Integrador 1 SCV0

Estudiantes:

Maicol Tadeo Carvajal Vargas

Michael Ramírez González

Ignacio Gabriel Vila Silva

Universidad CENFOTEC

Profe. Verónica Isabel Mora Lezcano

Bitácora 1

Descripción General del Sistema	3
Perfiles de Usuario y Actores Involucrados	3
Usuario Visitante (Estudiante / Público General)	3
Usuario Administrador (Comité Organizador)	3
Requisitos Funcionales	4
Módulo 1: Portal Público (Etapa 1 — Frontend)	4
Módulo 2: Lógica de Negocio y Administración (Etapa 2 — Backend)	5
Módulo 3: Experiencia de Usuario Multidispositivo	6
Requisitos No Funcionales	8
Suposiciones	10
Dependencias	11
Restricciones Generales	12
Restricciones de Diseño e Implementación	13
1. Tecnologías Obligatorias	13
2. Plataformas Soportadas	14
3. Normativas y Estándares de Código	14
3.1 Convenciones de Nomenclatura	14
3.2 Formato y Estilo de Código	15
3.3 Estructura del Proyecto	16
4. Estrategia de Branches (Git Flow Simplificado)	16
4.1 Ramas Principales (Permanentes)	16
4.2 Ramas de Trabajo (Temporales)	16
4.3 Flujo de Trabajo	17
5. Convención de Commits (Conventional Commits)	17
5.1 Tipos de Commit Permitidos	18
5.2 Reglas de Redacción	18
5.3 Ejemplos Válidos	18
5.4 Ejemplos Inválidos (a evitar)	19
6. Estándares Adicionales	19
Matriz de Trazabilidad	20
Requisitos Funcionales	20
Requisitos No Funcionales	23
Restricciones Cubiertas por Estándares	24

Descripción General del Sistema

Perfiles de Usuario y Actores Involucrados

Usuario Visitante (Estudiante / Público General)

Es el usuario final que asiste o desea participar en el festival. Su interacción con el sistema es principalmente de consumo de información e interacción básica (lectura y envío de formularios).

- **Objetivo principal:** Informarse sobre el evento, consultar la agenda, conocer los stands e inscribirse en las actividades de su interés.
- **Acciones en el sistema:**
 - Visualizar la página de inicio, información general, fechas y lugar del festival.
 - Navegar por el catálogo visual de actividades y ver el detalle de cada una.
 - Consultar la agenda cronológica y el estado de los cupos (simulado en Etapa 1).
 - Ver el directorio de stands, grupos o clubes participantes.
 - Llenar y enviar el formulario de inscripción a actividades (con validación del lado del cliente).
 - Enviar consultas a través de la página de contacto.

Usuario Administrador (Comité Organizador)

Es el miembro del equipo del CampusFest encargado de la logística, control y actualización de los datos del evento.

- **Objetivo principal:** Centralizar la gestión del festival para eliminar el uso de hojas de cálculo y registros manuales, evitando sobrecupos o duplicados.
- **Acciones en el sistema:**
 - **Gestión de Actividades:** Registrar nuevas actividades, editar la información existente o cancelarlas (lo que actualizará el estado en la agenda).
 - **Gestión de Participantes e Inscripciones:** Consultar la lista de estudiantes registrados y controlar el flujo de inscripciones.
 - **Gestión de Stands:** Registrar y organizar los datos de los stands, grupos o clubes.
 - **Control de Resultados:** Actualizar reconocimientos o resultados finales de las actividades (culturales, deportivas, etc.).

Requisitos Funcionales

Módulo 1: Portal Público (Etapa 1 — Frontend)

- **RF-01: Página de Inicio e Identidad del Festival** (→ CG3-1)

Descripción: El sistema debe contar con una página de inicio que presente la información general del festival y permita navegar hacia las secciones principales del sitio.

Criterio de aceptación (medible): La página de inicio muestra obligatoriamente: nombre del festival, descripción breve, fecha general, lugar principal, una sección destacada con tres (3) actividades principales, y al menos dos enlaces visibles que dirijan a las páginas de Actividades e Inscripción.

- **RF-02: Catálogo Visual de Actividades** (→ CG3-2)

Descripción: El sistema debe presentar un catálogo visual con todas las actividades del festival mediante tarjetas individuales.

Criterio de aceptación (medible): Cada tarjeta muestra los siguientes datos: nombre, categoría, fecha, hora, lugar y cupo disponible, e incluye un botón "Ver detalle" que dirige a la página de detalle de la actividad correspondiente.

- **RF-03: Detalle de la Actividad** (→ CG3-3)

Descripción: El sistema debe contar con una página dedicada que muestre la información completa de una actividad seleccionada.

Criterio de aceptación (medible): La página despliega los siguientes campos: nombre, descripción completa, categoría, fecha y hora, lugar, cupo máximo y requisitos de participación, junto con un botón de inscripción que dirige al formulario precargado con la actividad seleccionada.

- **RF-04: Agenda Dinámica del Festival** (→ CG3-4)

Descripción: El sistema debe mostrar la agenda completa del festival, organizando las actividades cronológicamente y permitiendo distinguir entre eventos pasados, en ejecución y futuros.

Criterio de aceptación (medible): La agenda lista las actividades ordenadas por fecha y hora, mostrando para cada una su nombre, hora, lugar, categoría y estado (disponible, lleno o cancelado). Los eventos clasificados como "próximos" cumplen con un margen mínimo de 8 horas respecto al momento de la consulta, y al hacer clic sobre una actividad se accede a su detalle completo mediante pop-up o página nueva.

- **RF-05: Directorio de Stands o Grupos** (→ CG3-5)

Descripción: El sistema debe mostrar la información de los stands, grupos o clubes participantes del festival con carácter meramente informativo.

Criterio de aceptación (medible): Cada stand muestra los siguientes datos: nombre, categoría, responsable, ubicación y descripción breve. El visitante no puede interactuar con los stands más allá de la consulta (sin votación, comentarios ni galería).

- **RF-06: Sección de Contacto y Preguntas Frecuentes** (→ CG3-6)

Descripción: El sistema debe ofrecer una página de contacto que permita al visitante conocer los canales de comunicación con el comité organizador y consultar las dudas más frecuentes.

Criterio de aceptación (medible): La página despliega obligatoriamente los siguientes elementos: información del comité organizador, correo de contacto, teléfono y formulario de consulta visual. Adicionalmente incluye una sección de preguntas frecuentes con al menos cinco preguntas y respuestas.

- **RF-07: Formulario de Inscripción con Validaciones** (→ CG3-7)

Descripción: El sistema debe proveer un formulario de inscripción a actividades del festival con validaciones del lado del cliente antes del envío.

Criterio de aceptación (medible): El formulario contiene siete (7) campos: nombre completo, identificación, correo, teléfono, carrera o grupo, actividad seleccionada y comentarios opcionales. Las validaciones JavaScript bloquean el envío si: (a) algún campo obligatorio está vacío, (b) el correo no cumple con el formato estándar (texto@texto.dominio), (c) el teléfono está vacío, o (d) no se ha seleccionado una actividad. Al completar correctamente, el sistema muestra una confirmación visual en pantalla, sin envío de correo ni generación de QR.

Módulo 2: Lógica de Negocio y Administración (Etapa 2 — Backend)

- **RF-08: Acceso Diferenciado por Correo Institucional** (→ CG3-13)

Descripción: El sistema debe diferenciar el contenido visible según el usuario haya o no iniciado sesión, sin usar contraseñas ni autenticación tradicional.

Criterio de aceptación (medible): Cualquier persona que ingrese al sitio sin iniciar sesión accede como Visitante. Al ingresar un correo electrónico institucional válido en el campo correspondiente, el sistema reconoce al usuario como Administrador y le habilita la visualización del detalle completo del festival y las funciones de gestión. No se solicitan ni almacenan contraseñas en ningún punto del flujo.

- **RF-09: Control de Duplicados y Gestión Automática de Cupos** (→ CG3-8)

Descripción: El sistema debe prevenir inscripciones duplicadas en una misma actividad y mantener actualizado el cupo disponible de cada actividad de forma automática.

Criterio de aceptación (medible): Antes de registrar una inscripción, el sistema verifica que el participante (identificado por su cédula o correo) no esté previamente inscrito en esa misma actividad; en caso de duplicado, bloquea la inscripción y muestra un mensaje de

error. Tras cada inscripción exitosa, el cupo disponible se decrementa en uno (1); al llegar a cero (0), la actividad cambia automáticamente a estado "lleno" y el formulario queda bloqueado para esa actividad específica. Un mismo participante sí puede inscribirse en actividades diferentes.

- **RF-10: Gestión de Actividades (CRUD)** (→ CG3-9)

Descripción: El sistema debe permitir al usuario Administrador crear, consultar, modificar y cancelar actividades del festival.

Criterio de aceptación (medible): El Administrador puede ejecutar las cuatro (4) operaciones (registrar, consultar, editar y cancelar) sobre cualquier actividad. Al registrar o editar, el sistema valida que todos los campos obligatorios (nombre, categoría, fecha, hora, lugar, cupo máximo, descripción y requisitos) estén completos antes de persistir en MongoDB. Las actividades canceladas se reflejan automáticamente en la agenda con el estado correspondiente.

- **RF-11: Administración de Stands y Control de Participantes** (→ CG3-10)

Descripción: El sistema debe permitir al Administrador registrar stands del festival y consultar la lista de participantes inscritos por actividad.

Criterio de aceptación (medible): El Administrador puede registrar un nuevo stand completando los cinco (5) campos obligatorios: nombre, categoría, responsable, ubicación y descripción. Adicionalmente, puede consultar para cada actividad la lista de participantes inscritos, mostrando al menos tres datos por participante: nombre completo, identificación y correo.

- **RF-12: Publicación de Resultados y Reconocimientos** (→ CG3-11)

Descripción: El sistema debe permitir al Administrador actualizar el estado final y los resultados de una actividad, y visibilizar esa información al público en la misma sección de Actividades.

Criterio de aceptación (medible): Desde la vista de gestión, el Administrador puede actualizar el estado de la actividad (finalizada, suspendida, etc.) y agregar los resultados o reconocimientos correspondientes. Una vez publicados, los resultados se muestran al Visitante directamente en la vista de detalle de la actividad, sin requerir navegar a otra página.

Módulo 3: Experiencia de Usuario Multidispositivo

- **RF-13: Diseño Adaptable Responsive** (→ CG3-12)

Descripción: El sistema debe adaptar su interfaz a diferentes tamaños de pantalla sin comprometer la legibilidad ni la usabilidad.

Criterio de aceptación (medible): En pantallas de escritorio, el menú se muestra horizontalmente, las tarjetas se distribuyen en múltiples columnas y los formularios quedan centrados. En pantallas móviles, el menú se reorganiza o colapsa, las tarjetas se apilan en

una sola columna, los formularios ocupan el 100% del ancho disponible y los botones mantienen un tamaño mínimo de 44×44 px para facilitar su pulsación. Se implementan media queries propias para gestionar estas adaptaciones.

- **RF-14: Modo Oscuro Automático** (→ CG3-12)

Descripción: El sistema debe ajustar automáticamente su tema visual (claro u oscuro) según la preferencia del sistema operativo del usuario.

Criterio de aceptación (medible): El sitio detecta mediante la media query CSS `prefers-color-scheme` la configuración de tema del usuario y aplica los estilos correspondientes sin necesidad de interruptores manuales ni recarga de la página. La implementación se realiza con media queries propias en CSS nativo, sin depender de variables de frameworks externos.

Requisitos No Funcionales

- **RNF-01: Rendimiento (Tiempo de Carga)**

Descripción: El sistema debe responder con agilidad para no afectar la experiencia del visitante durante el festival.

Métrica (medible): El tiempo de carga inicial de las páginas de Inicio, Catálogo de Actividades y Agenda no debe superar los 2.5 segundos en una conexión 4G estándar (≈ 10 Mbps).

- **RNF-02: Usabilidad (Accesibilidad Básica)**

Descripción: La interfaz debe ser legible y operable tanto en escritorio como en móvil, considerando criterios mínimos de accesibilidad.

Métrica (medible): Todos los textos mantienen una relación de contraste mínima de 4.5:1 (WCAG 2.1 nivel AA). Todos los elementos interactivos (botones, enlaces, campos) cuentan con etiquetas descriptivas y son alcanzables mediante navegación por teclado (tabulación).

- **RNF-03: Compatibilidad (Navegadores)**

Descripción: El sitio web debe funcionar correctamente en los navegadores modernos más utilizados.

Métrica (medible): El sistema se prueba y mantiene compatible con las dos versiones más recientes de Google Chrome, Mozilla Firefox y Microsoft Edge, sin pérdida de funcionalidades visibles.

- **RNF-04: Mantenibilidad (Calidad del Código)**

Descripción: El código fuente debe seguir convenciones claras y consistentes para facilitar el mantenimiento futuro y el trabajo colaborativo.

Métrica (medible): El 100% del código entregado respeta las convenciones de nomenclatura, formato e indentación definidas en la sección de Restricciones de Implementación; el repositorio cumple además con la estrategia de branches y la convención de commits acordada por el equipo.

- **RNF-05: Disponibilidad**

Descripción: El portal debe permanecer accesible durante el período de operación del festival.

Métrica (medible): El sistema garantiza una disponibilidad mínima del 99% durante los siete (7) días previos y los días de ejecución del festival, lo que equivale a un máximo permitido de 1.7 horas acumuladas de inactividad por semana.

- **RNF-06: Integridad de Datos (Validación)**

Descripción: El sistema debe asegurar la integridad de los datos almacenados validando tanto en cliente como en servidor.

Métrica (medible): El 100% de los formularios valida los datos primero en el navegador (JavaScript) y nuevamente en el servidor (Node.js/Express) antes de persistir en MongoDB. No se aceptan registros que incumplan los formatos definidos para correo, identificación y teléfono.

Suposiciones

El equipo de desarrollo parte de los siguientes supuestos al momento de elaborar la presente especificación:

- **Disponibilidad de información del cliente:** Se asume que la institución educativa proporcionará oportunamente los contenidos reales del festival (nombres de actividades, descripciones, stands, responsables, fechas y lugares), así como el material visual oficial (logotipo, paleta de colores, manual de marca) referenciado por el cliente.
- **Acceso a la red:** Se asume que tanto los visitantes como los miembros del comité organizador contarán con acceso a internet desde sus dispositivos (computadora o teléfono móvil) durante el período de uso del sistema.
- **Confianza en el correo institucional:** Dado que el cliente decidió no implementar mecanismos de autenticación con contraseña, se asume que el ingreso del correo institucional como mecanismo de identificación es suficiente para reconocer a los miembros del comité organizador en el contexto del festival.
- **Volumen de datos contenido:** Se asume que el sistema operará con un volumen acotado de información (decenas de actividades, cientos de inscripciones), correspondiente a la escala de un festival estudiantil interno, sin requerir optimizaciones avanzadas de escalabilidad.
- **Navegadores actualizados:** Se asume que los usuarios finales utilizarán versiones recientes de navegadores modernos compatibles con HTML5, CSS3 y ES6+.

Dependencias

El correcto funcionamiento del sistema depende de los siguientes elementos externos al desarrollo:

- **Entorno de ejecución Node.js:** El backend de la Etapa 2 depende de Node.js (versión LTS 20.x o superior) y del gestor de paquetes npm para la instalación de las librerías del proyecto.
- **Base de datos MongoDB:** El sistema depende de una instancia activa de MongoDB, ya sea local o alojada en MongoDB Atlas, para la persistencia de las actividades, inscripciones, stands y resultados.
- **Plataforma de control de versiones:** El desarrollo colaborativo depende de la disponibilidad de una plataforma Git (GitHub o equivalente) para la gestión del repositorio, las ramas y los pull requests.
- **Plataforma de gestión de proyecto:** El seguimiento de las historias de usuario, sprints y trazabilidad depende del acceso al proyecto en Jira por parte de todos los miembros del equipo y del docente.
- **Conectividad del usuario final:** El sistema depende de una conexión a internet estable por parte del usuario para acceder al portal y completar inscripciones.

Restricciones Generales

El sistema deberá desarrollarse respetando las siguientes limitaciones obligatorias:

- **Stack tecnológico cerrado:** El desarrollo está restringido al uso de **JavaScript (vanilla) en frontend, Node.js + Express en backend y MongoDB como base de datos**. No se permiten lenguajes adicionales (PHP, Python, Java, etc.) ni bases relacionales (MySQL, PostgreSQL, etc.).
- **Sin autenticación tradicional:** El sistema **no debe almacenar, transmitir ni gestionar contraseñas** de ningún tipo. La diferenciación entre Visitante y Administrador se realiza únicamente mediante el ingreso de un correo institucional.
- **Sin notificaciones externas:** El sistema **no debe enviar correos electrónicos, mensajes SMS ni notificaciones push**. Todas las confirmaciones se realizan exclusivamente dentro del sitio web.
- **Modo oscuro con CSS nativo:** La implementación del modo oscuro debe realizarse mediante **media queries propias en CSS nativo** utilizando prefers-color-scheme, sin depender de variables o componentes de frameworks externos.
- **Validación dual obligatoria:** Toda la información ingresada por el usuario debe validarse tanto en el cliente (JavaScript) como en el servidor (Express) antes de persistir en MongoDB.
- **Plazo de entrega:** El desarrollo se ejecuta dentro del cronograma académico definido para el curso, con entregas parciales y final según el calendario establecido por la institución.

Restricciones de Diseño e Implementación

1. Tecnologías Obligatorias

El sistema debe desarrollarse utilizando exclusivamente el siguiente stack tecnológico, alineado con las restricciones del curso y del cliente:

Capa	Tecnología
Lenguaje principal	JavaScript (ES6+)
Frontend — Estructura	HTML5
Frontend — Estilos	CSS3 (nativo, con media queries propias)
Frontend — Lógica	JavaScript Vanilla
Backend — Entorno	Node.js
Backend — Framework	Express.js
Base de datos	MongoDB (local o Atlas)
Control de versiones	Git
Plataforma de repositorio	GitHub
Gestión de proyecto	Jira (Scrum)

Prohibiciones explícitas:

- No se permite el uso de frameworks de frontend (React, Vue, Angular, Svelte).
- No se permiten preprocesadores de CSS (Sass, Less) ni frameworks de estilos (Bootstrap, Tailwind, Materialize) para la implementación del modo oscuro; este debe ser CSS nativo.
- No se permite el uso de bases de datos relacionales.
- No se permiten librerías de autenticación (Passport, JWT, OAuth) ya que el sistema no maneja contraseñas.

2. Plataformas Soportadas

- **Navegadores de escritorio:** Últimas dos versiones estables de Google Chrome, Mozilla Firefox y Microsoft Edge.
- **Navegadores móviles:** Últimas dos versiones estables de Chrome Mobile y Safari iOS.
- **Resoluciones soportadas:** Desde 320 px (móviles pequeños) hasta 1920 px (escritorio Full HD).
- **Sistemas operativos:** Independiente del SO del usuario final, ya que el sistema es 100% web.

3. Normativas y Estándares de Código

3.1 Convenciones de Nomenclatura

Elemento	Convención	Ejemplo
Variables y funciones (JS)	camelCase	cuposDisponibles, validarFormulario()
Constantes (JS)	UPPER_SNAKE_CASE	MAX_CUPOS_ACTIVIDAD, API_BASE_URL
Clases / Modelos	PascalCase	Actividad, Inscripcion, Stand
Archivos JS	kebab-case.js	formulario-inscripcion.js, detalle-actividad.js
Archivos HTML	kebab-case.html	detalle-actividad.html, agenda.html
Archivos CSS	kebab-case.css	estilos-globales.css, modo-oscuro.css
Carpetas	kebab-case (minúsculas)	public/, routes/, controllers/
Clases CSS	kebab-case	.tarjeta-actividad, .boton-inscripcion
IDs HTML	kebab-case	id="formulario-inscripcion"

Elemento	Convención	Ejemplo
Atributos name en formularios	snake_case	name="correo_institucional"
Colecciones MongoDB	Plural en minúsculas	actividades, inscripciones, stands
Campos de documentos MongoDB	camelCase	nombreCompleto, fechaInscripcion
Endpoints REST	kebab-case plural	/api/actividades, /api/inscripciones

Idioma: Todo el código (variables, funciones, comentarios, commits, nombres de ramas) se redacta en **español** para mantener coherencia con el dominio del proyecto.

3.2 Formato y Estilo de Código

- **Indentación:** 2 espacios (no tabulaciones).
- **Final de archivo:** Toda la línea debe terminar con un salto de línea (LF, no CRLF).
- **Longitud máxima de línea:** 100 caracteres.
- **Comillas en JavaScript:** Comillas simples ' para strings; comillas dobles " solo cuando el string contiene una comilla simple.
- **Punto y coma:** Obligatorio al final de cada sentencia en JavaScript.
- **Llaves:** Estilo K&R (apertura en la misma línea de la declaración).
- **Espacios:** Un espacio antes y después de operadores (a = b + c, no a=b+c).
- **Comentarios:** Se utiliza // para comentarios de una línea y /* */ para bloques. Se documentan únicamente decisiones no evidentes o lógica compleja; no se comentan líneas obvias.
- **Funciones:** Máximo 30 líneas por función; si excede, se debe refactorizar.
- **Sin código muerto:** No se permiten `console.log()` ni variables sin uso en código entregado a la rama main.

3.3 Estructura del Proyecto

```
campusfest/
├── public/                                # Archivos estáticos (frontend)
│   ├── css/
│   │   ├── estilos-globales.css
│   │   └── modo-oscuro.css
│   ├── js/
│   │   ├── formulario-inscripcion.js
│   │   └── agenda.js
│   ├── img/
│   └── pages/
│       ├── index.html
│       ├── actividades.html
│       ├── detalle-actividad.html
│       ├── agenda.html
│       ├── stands.html
│       ├── contacto.html
│       └── inscripcion.html
├── src/                                  # Backend (Etapa 2)
│   ├── controllers/
│   ├── models/
│   ├── routes/
│   ├── middlewares/
│   └── config/
├── docs/                                # Documentación (ERS, bitácoras)
├── .gitignore
├── package.json
├── server.js
└── README.md
```

4. Estrategia de Branches (Git Flow Simplificado)

4.1 Ramas Principales (Permanentes)

Rama	Propósito	Reglas
main	Código en producción, estable y entregable	Protegida. Solo recibe merges desde develop mediante Pull Request aprobado. No se permiten commits directos.
develop	Rama de integración del trabajo en curso	Recibe merges desde las ramas de tipo feature/, fix/, docs/.

4.2 Ramas de Trabajo (Temporales)

Se crean a partir de **develop** y se eliminan tras el merge:

Prefijo	Uso	Ejemplo
feature/	Nueva funcionalidad o historia de usuario	feature/CG3-7-formulario-inscripcion
fix/	Corrección de errores	fix/CG3-8-validacion-cupo-cero
docs/	Cambios solo en documentación	docs/actualizar-ers-suposiciones
refactor/	Refactorización sin cambio funcional	refactor/CG3-4-componentes-agenda
style/	Ajustes de formato o estilos visuales	style/CG3-12-media-queries-mobile

Convención de nombres de rama: tipo/CG3-XX-descripcion-corta-en-kebab-case

- Cada rama de feature/fix se asocia al código de la historia de Jira correspondiente (CG3-XX).
- La descripción es breve (máximo 5 palabras), en minúsculas y con guiones.

4.3 Flujo de Trabajo

1. El desarrollador crea su rama a partir de develop: `git checkout develop && git pull && git checkout -b feature/CG3-7-formulario-inscripcion`
2. Trabaja localmente con commits frecuentes siguiendo la convención (sección 5).
3. Cuando termina, abre un Pull Request hacia develop.
4. Al menos otro miembro del equipo revisa y aprueba.
5. Tras la aprobación, se realiza merge (preferentemente squash and merge) y se elimina la rama.
6. Al cierre de cada sprint, develop se mergea a main y se crea un tag de versión (v1.0.0, v1.1.0, etc.).

5. Convención de Commits (Conventional Commits)

Todos los commits deben seguir el formato:

<tipo>(<alcance opcional>): <descripción breve en español>

[cuerpo opcional]

[footer opcional con referencia a Jira]

5.1 Tipos de Commit Permitidos

Tipo	Uso
feat	Nueva funcionalidad o historia implementada
fix	Corrección de un bug o error
docs	Cambios únicamente en documentación
style	Cambios de formato, espacios, indentación (sin afectar lógica)
refactor	Reorganización de código sin cambiar comportamiento
test	Agregar o modificar pruebas
chore	Tareas de mantenimiento, configuración, dependencias
perf	Mejoras de rendimiento

5.2 Reglas de Redacción

- La descripción comienza con un verbo en **modo imperativo y en español** ("agrega", "corrige", "actualiza"), no en pasado.
- Primera letra en minúscula.
- Sin punto al final.
- Máximo 72 caracteres en la línea de título.
- El cuerpo (opcional) explica el "por qué", no el "qué".
- Se incluye el código de la historia de Jira en el footer cuando aplica.

5.3 Ejemplos Válidos

`feat(inscripcion): agrega validacion de correo institucional`

Implementa la validacion en JavaScript para verificar formato de correo antes de habilitar el envio del formulario.

Refs: CG3-7

`fix(agenda): corrige filtro de eventos pasados`

Refs: CG3-4

`docs(ers): actualiza matriz de trazabilidad con CG3-13`

`style(mobile): ajusta tarjetas a una columna en pantallas pequenas`

Refs: CG3-12

5.4 Ejemplos Inválidos (a evitar)

- “actualice cosas” (sin tipo, vago)
- “feat: Cambios” (descripción no descriptiva)
- “FIX: Agregado el formulario”. (mayúsculas, punto final, verbo en pasado)
- “feat(formulario): se agrego el campo de teléfono y se corrigió el bug del cupo”
(mezcla dos cambios → deben ser commits separados)

6. Estándares Adicionales

- Accesibilidad: Cumplimiento mínimo del nivel WCAG 2.1 AA en contraste de colores y navegación por teclado.
- README obligatorio: El repositorio debe incluir un README.md con: descripción del proyecto, instrucciones de instalación, scripts disponibles y miembros del equipo.
- Archivo .gitignore: Debe excluir node_modules/, archivos de entorno (.env), archivos del IDE (.vscode/, .idea/) y archivos del sistema (.DS_Store).
- Variables de entorno: Las credenciales de MongoDB y demás configuraciones sensibles se gestionan mediante un archivo .env (excluido del repositorio) y se documentan en un .env.example.

Matriz de Trazabilidad

La presente matriz vincula cada requisito definido en la ERS con la historia de usuario correspondiente en Jira, el módulo del sistema al que pertenece, la prueba que verifica su cumplimiento y la evidencia esperada al cierre de la implementación.

Requisitos Funcionales

ID Requisito	Nombre del Requisito	Historia Jira	Módulo / Etapa	Prueba de Verificación	Evidencia Esperada
RF-01	Página de Inicio e Identidad del Festival	CG3-1	Portal Público / Etapa 1	Inspección visual: verificar que la página de inicio contenga nombre, descripción, fecha, lugar, 3 actividades destacadas y enlaces a Actividades e Inscripción.	Archivo index.html desplegado; captura de pantalla de la home.
RF-02	Catálogo Visual de Actividades	CG3-2	Portal Público / Etapa 1	Verificar que cada tarjeta del catálogo contenga los 6 datos obligatorios y un botón funcional "Ver detalle".	Archivo actividades.html; captura mostrando al menos 3 tarjetas con datos completos.
RF-03	Detalle de la Actividad	CG3-3	Portal Público / Etapa 1	Acceder al detalle desde el catálogo y validar la presencia de los 7 campos obligatorios y el botón de inscripción.	Archivo detalle-actividad.html; captura del detalle de una actividad.
RF-04	Agenda Dinámica del Festival	CG3-4	Portal Público / Etapa 1	Validar que las actividades se muestren	Archivo agenda.html; captura

ID Requisito	Nombre del Requisito	Historia Jira	Módulo / Etapa	Prueba de Verificación	Evidencia Esperada
				ordenadas cronológicamente, con estado visible, filtro de 8 horas y acceso al detalle desde la agenda.	mostrando eventos pasados, en ejecución y futuros con sus estados.
RF-05	Directorio de Stands o Grupos	CG3-5	Portal Público / Etapa 1	Inspección visual: verificar la presencia de los 5 datos obligatorios por stand y la ausencia de elementos interactivos.	Archivo stands.html; captura del directorio.
RF-06	Sección de Contacto y Preguntas Frecuentes	CG3-6	Portal Público / Etapa 1	Verificar la presencia de los 4 elementos obligatorios más al menos 5 preguntas frecuentes.	Archivo contacto.html; captura de la sección de FAQ.
RF-07	Formulario de Inscripción con Validaciones	CG3-7	Portal Público / Etapa 1	Pruebas funcionales sobre el formulario: enviar vacío (debe bloquear), correo mal formado (debe bloquear), datos correctos (debe mostrar confirmación).	Archivo inscripcion.html; capturas de los 3 escenarios de validación.
RF-08	Control de Duplicados y Gestión Automática de Cupos	CG3-8	Backend / Etapa 2	Intentar inscribirse dos veces a la misma persona en la misma actividad (debe bloquear).	Capturas del mensaje de duplicado y del estado "lleno"; documento en

ID Requisito	Nombre del Requisito	Historia Jira	Módulo / Etapa	Prueba de Verificación	Evidencia Esperada
				Inscribir hasta llenar el cupo y validar bloqueo automático.	MongoDB con cupo en 0.
RF-09	Gestión de Actividades (CRUD)	CG3-9	Backend / Etapa 2	Ejecutar las 4 operaciones (crear, leer, editar, cancelar) y validar campos obligatorios al guardar.	Capturas de cada operación; registros persistidos en la colección actividades de MongoDB.
RF-10	Administración de Stands y Control de Participantes	CG3-10	Backend / Etapa 2	Registrar un stand con sus 5 campos. Consultar la lista de participantes inscritos en una actividad.	Capturas del formulario de stand y del listado de participantes; registros en MongoDB.
RF-11	Publicación de Resultados y Reconocimientos	CG3-11	Backend / Etapa 2	El Administrador actualiza estado y resultados de una actividad; el Visitante los visualiza en el detalle.	Captura del panel de actualización y del detalle público de la actividad con resultados visibles.
RF-12	Diseño Adaptable Responsive	CG3-12	UX Multidispositivo / Etapa 1	Probar el sitio en resoluciones de 320 px, 768 px y 1440 px. Validar menú, tarjetas, formularios y botones.	Capturas comparativas escritorio vs. móvil de las páginas clave.
RF-13	Modo Oscuro Automático	CG3-12	UX Multidispositivo / Etapa 1	Cambiar el tema del SO entre claro y oscuro; verificar que el sitio responda	Capturas del sitio en modo claro y oscuro; fragmento del CSS con

ID Requisito	Nombre del Requisito	Historia Jira	Módulo / Etapa	Prueba de Verificación	Evidencia Esperada
				sin recarga ni interruptor.	prefers-color-scheme.
RF-14	Acceso Diferenciado por Correo Institucional	CG3-13	Backend / Etapa 2	Acceder sin correo (debe ver vista Visitante); ingresar correo institucional válido (debe ver vista Administrador).	Capturas de ambas vistas; verificación de que no exista campo de contraseña en el flujo.

Requisitos No Funcionales

ID Requisito	Nombre del Requisito	Atributo de Calidad	Verificación	Evidencia Esperada
RNF-01	Rendimiento (Tiempo de Carga)	Performance	Medición con Chrome DevTools (pestaña Network y Lighthouse) en conexión simulada 4G.	Reporte de Lighthouse mostrando FCP $\leq$ 2.5 s en las 3 páginas clave.
RNF-02	Usabilidad (Accesibilidad Básica)	Usabilidad / Accesibilidad	Auditoría con Lighthouse Accessibility y navegación manual por teclado (Tab).	Reporte Lighthouse con score $\geq$ 90 en accesibilidad; video o captura de navegación por teclado.
RNF-03	Compatibilidad (Navegadores)	Portabilidad	Pruebas exploratorias en Chrome, Firefox y Edge (últimas 2 versiones).	Capturas del sitio funcionando en los 3 navegadores.
RNF-04	Mantenibilidad (Calidad del Código)	Mantenibilidad	Inspección manual del repositorio: nomenclatura, branches y commits.	Historial de Git mostrando ramas y commits

ID Requisito	Nombre del Requisito	Atributo de Calidad	Verificación	Evidencia Esperada
				conformes a la convención.
RNF-05	Disponibilidad	Confiabilidad	Monitoreo de uptime del servicio durante el período del festival.	Reporte de disponibilidad del proveedor de hosting.
RNF-06	Integridad de Datos (Validación)	Confiabilidad / Integridad	Enviar datos inválidos por API (Postman) con el cliente bypassado; el servidor debe rechazarlos.	Capturas de Postman con respuesta 400 ante datos mal formados.

Restricciones Cubiertas por Estándares

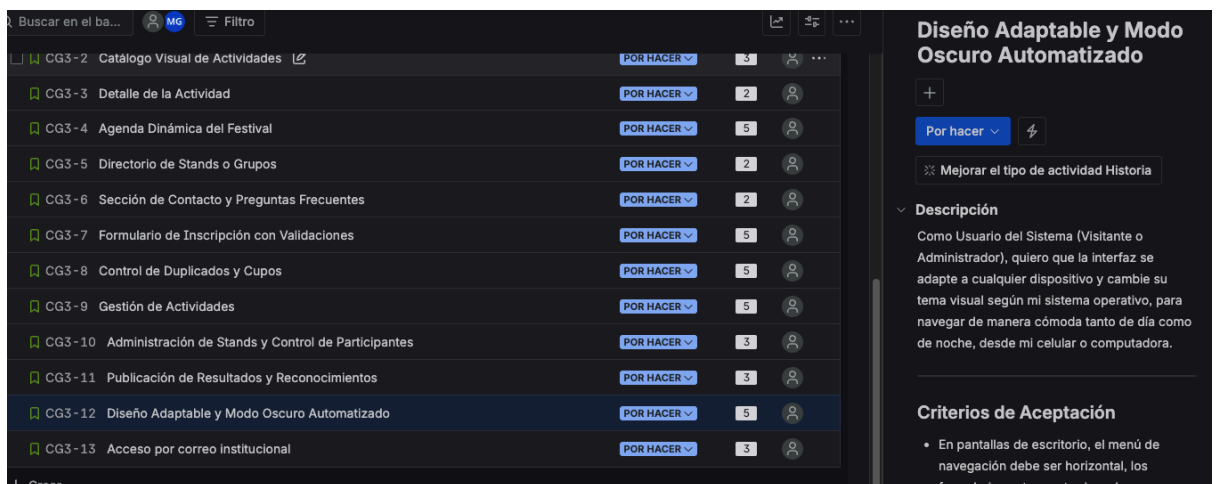
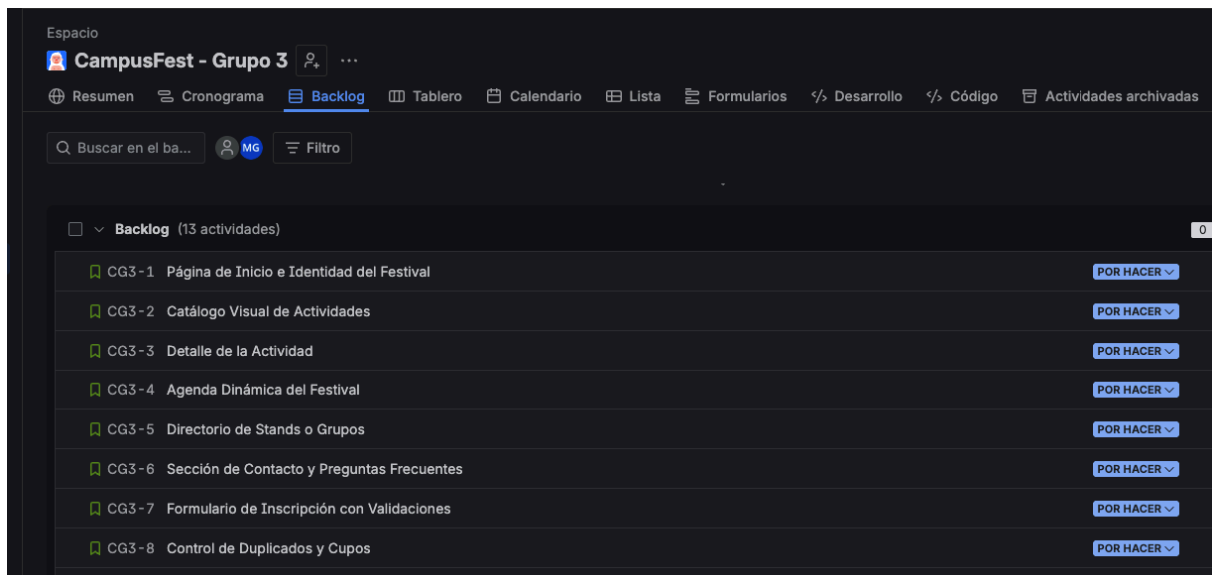
ID Restricción	Descripción	Verificación	Evidencia Esperada
REST-01	Stack tecnológico (JS, Node, Express, MongoDB)	Inspección de package.json y código fuente.	Archivo package.json con dependencias permitidas.
REST-02	Sin autenticación con contraseña	Inspección de modelos y endpoints.	Ausencia de campos password o similares en la base de datos.
REST-03	Modo oscuro con CSS nativo	Inspección de archivos CSS.	Fragmento de CSS con @media (prefers-color-scheme: dark).
REST-04	Convenciones de nomenclatura	Inspección del repositorio.	Estructura de carpetas y nombres conformes a la sección 3.1.
REST-05	Estrategia de branches	Inspección del repositorio en GitHub.	Capturas del árbol de ramas mostrando main, develop y feature/*.

ID Restricción	Descripción	Verificación	Evidencia Esperada
REST-06	Convención de commits	Inspección del historial Git.	Historial mostrando commits conformes a Conventional Commits.

Actividades realizadas

Desarrollo del Sprint 1 (Duración: Semanas 1 a 4) * Objetivos del Sprint: Educción de requerimientos, creación de la ERS y configuración inicial de Jira.

- Roles del Equipo:
 - * Ignacio Gabriel Vila: Scrum Master / Analista de Requerimientos (Redacción de RNF, Suposiciones y Restricciones).
 - * Maicol Tadeo Carvajal: Administrador de Jira / Diseñador de Arquitectura (Configuración del proyecto Scrum, asignación de Story Points y matriz).
 - * Michael Ramírez: Analista de Requerimientos / Product Owner (Definición de Perfiles de Usuario y Requisitos Funcionales).
- Evidencia Visual:



Reporte de inconvenientes y soluciones

Inconveniente 1: Delimitación de Criterios de Aceptación Medibles

* *Descripción:* Al inicio fue complejo redactar los criterios de aceptación de los módulos funcionales de manera que fueran 100% medibles para las pruebas de software.

* *Solución:* Se aplicó el enfoque de validaciones del lado del cliente y del servidor (RF-07 y RNF-06), acotando numéricamente los campos y respuestas esperadas.

Inconveniente 2: Gestión de Roles sin Autenticación Tradicional

* *Descripción:* Diseñar la lógica de negocio para el acceso de Administrador basándose únicamente en el correo institucional (restringido por el cliente) planteó dudas sobre el control de flujo.

* *Solución:* Se resolvió a nivel de requerimiento mediante una arquitectura de rutas diferenciadas en el backend (Express) que evalúa el dominio del correo antes de renderizar las vistas de gestión (RF-08).

Enlaces de Acceso

* **Enlace del Repositorio (GitHub):**

<https://github.com/mramirezg7/ProyIntegrador1Grupo3>

* **Enlace del Proyecto en Jira:**

<https://practicasmanauno.atlassian.net/jira/software/projects/CG3/boards/68/backlog>